BRAINERX · AI MASTERCLASS

Ensemble Learning

Classification Metrics · Decision Trees · Random Forest · XGBoost

What We'll Cover Today

A full deep-dive into classification evaluation, trees, and ensemble methods

01 **Classification Evaluation**

Confusion Matrix, Precision, Recall, F1-Score — how to measure a model

02 **Decision Trees**

How trees think, Gini Impurity, Entropy, manual construction step-by-step

03 **Overfitting & Variance**

Why trees fail alone and why we need ensemble methods

04 **Ensemble Learning**

Bagging, Boosting, Stacking — the three pillars

05 **Random Forest**

Bootstrap sampling, feature randomness, feature importance

06 **XGBoost**

Sequential boosting, gradient correction, hyperparameters

07 **Model Comparison**

When to use which model and why

08 **Summary & Cheatsheet**

Everything on one page — formulas, rules, tips

01

Classification Evaluation

BrainerX · AI Masterclass · Session 7

Why Accuracy Alone Is NOT Enough

The most dangerous metric in Machine Learning

The Fraud Detection Problem

Imagine a fraud detection model:

- 100,000 transactions in the test set
- 99,800 are legitimate (normal)
- 200 are fraudulent (positive class)

A dumb model that predicts 'NOT FRAUD' for everything:
→ Accuracy = 99,800 / 100,000 = 99.8% ← looks amazing!

But it caught ZERO fraudulent transactions.
This model is completely useless — yet 99.8% accurate!

The Solution

We need metrics that reveal WHERE the model fails, not just overall %.

The tools we need:
→ Confusion Matrix
→ Precision
→ Recall (Sensitivity)
→ F1-Score

These tell us the REAL story.

💡 Rule: Always look beyond accuracy — especially when classes are imbalanced.

The Confusion Matrix

A 2×2 table that shows exactly what your model got right and wrong

	Predicted: YES	Predicted: NO
Actual: YES	**TP** Model said YES Actually YES ✓	**FP** Model said YES Actually NO ✗
Actual: NO	**FN** Model said NO Actually YES ✗	**TN** Model said NO Actually NO ✓

TP (True Positive)

You predicted fraud → it WAS fraud. Perfect!

TN (True Negative)

You predicted no fraud → it WASN'T fraud. Correct!

FP (False Positive)

You predicted fraud → it was actually normal. False alarm.

FN (False Negative)

You predicted no fraud → it WAS fraud. Missed it!

Confusion Matrix — Titanic Example

	Predicted: Survived	Predicted: Did Not
Actual: Survived	64 TP	25 FN
Actual: Did Not	16 FP	118 TN

Reading Our Results (223 test passengers):

TP = 64 → Model correctly predicted 64 survivors
TN = 118 → Model correctly predicted 118 non-survivors
FP = 16 → Model predicted survived, actually didn't
FN = 25 → Model missed 25 actual survivors

Accuracy = (64+118) / 223 = 81.6%

Total correct predictions: 182 out of 223

⚠ But we missed 25 actual survivors (FN)!
Is that acceptable? → Recall tells us.

Precision — Quality of Positive Predictions

When you say 'Yes', how often are you right?

$$\text{Precision} = TP / (TP + FP)$$

Plain English

Of ALL the passengers I predicted survived, what % actually did?

Titanic example:
Precision = 64 / (64 + 16) = 64/80 = 80%
→ 80% of predicted survivors actually survived

⚠ When Precision Matters Most

- Spam filter:
  Low precision = real emails go to spam!
  → Prioritize Precision

- Cancer treatment (expensive/painful):
  Low precision = treating healthy patients
  → Prioritize Precision

Precision Scale: 0% (all FP, useless) ←————————→ 100% (no FP, perfect)

Higher Precision = Fewer False Alarms

Recall (Sensitivity) — Coverage of Actual Positives

Of ALL real positives, how many did you actually catch?

$$\text{Recall} = TP / (TP + FN)$$

Plain English

Of ALL the passengers who ACTUALLY survived, how many did I predict?

Titanic example:
Recall = 64 / (64 + 25) = 64/89 = 71.9%
→ We caught 72% of real survivors
→ We MISSED 28% of survivors (FN=25)

⚠ When Recall Matters Most

Cancer screening (disease detection):
Missing a sick patient is catastrophic!
→ Prioritize Recall

Fraud detection (catching fraud):
Missing real fraud = financial loss
→ Prioritize Recall

FN = the cases you missed

Precision vs Recall Trade-off: Improving one often hurts the other → F1-Score balances both

F1-Score — The Balanced Metric

The single best metric for imbalanced classification problems

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

High Precision
Low Recall

P=0.95 R=0.30

F1 = 2×(0.95×0.30)/(0.95+0.30) = 0.456

Catches very few positives
but when it does, it's correct.
Not useful in most cases.

Low Precision
High Recall

P=0.30 R=0.95

F1 = 2×(0.30×0.95)/(0.30+0.95) = 0.456

Catches everything but also
lots of false alarms.
Also not ideal.

Balanced
Precision & Recall

P=0.80 R=0.80

F1 = 2×(0.80×0.80)/(0.80+0.80) = 0.800

Both precision and recall
are good. This model is
actually useful!

F1 penalizes extreme imbalance between Precision and Recall. A perfect score is 1.0. Minimum is 0.

Metrics Cheat Sheet — When to Use What

Choose your evaluation metric BEFORE training based on your problem type

Metric	Formula	Best When	Weakness
Accuracy	`(TP+TN)/Total`	Classes are balanced	Misleading when imbalanced
Precision	`TP/(TP+FP)`	FP is costly (spam filter)	Ignores False Negatives
Recall	`TP/(TP+FN)`	FN is costly (disease screen)	Ignores False Positives
F1-Score	`2×P×R/(P+R)`	Both FP and FN matter equally	Less interpretable to clients

💡 For the Titanic dataset: Use F1-Score because classes are slightly imbalanced (62% not survived, 38% survived).

02

Decision Trees

What Is a Decision Tree?

A supervised learning algorithm that makes decisions like a human flowchart

Supervised Learning

It learns from labeled training data (inputs + correct answers)

Classification & Regression

Can predict categories (survived/not) OR numbers (price)

Fully Interpretable

You can literally READ the tree and understand every decision

Hierarchical Structure

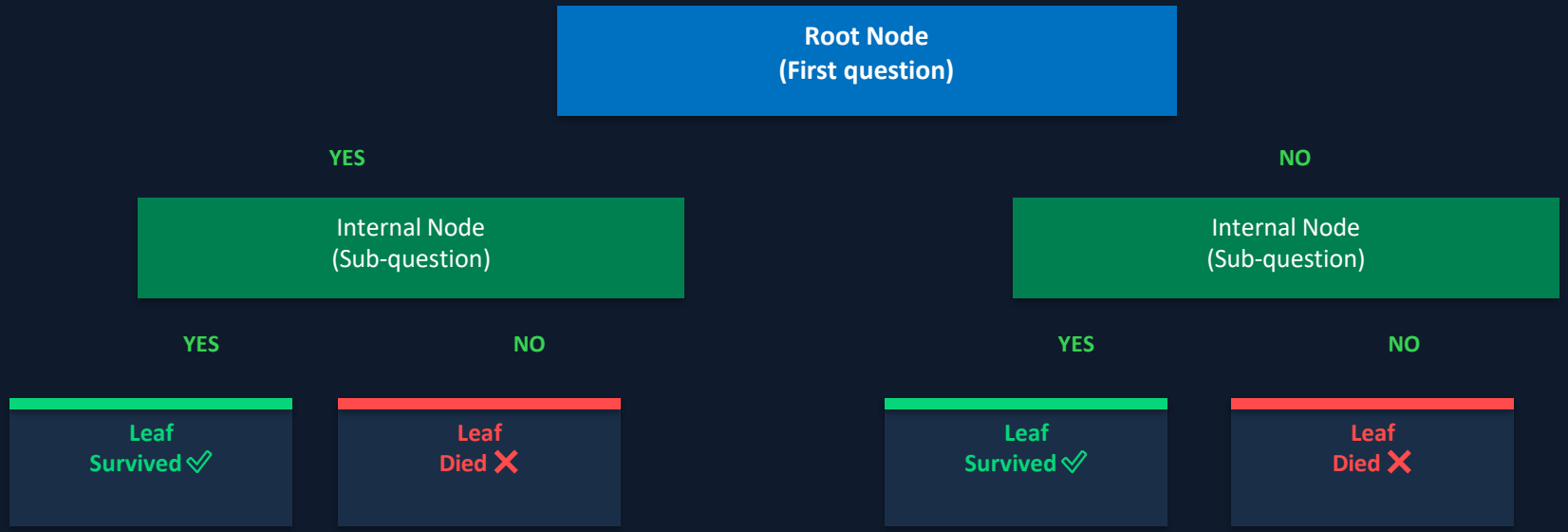
Root node → Internal nodes → Leaf nodes (final predictions)

Yes/No Questions

Each node splits data using a yes/no question on one feature

Anatomy of a Decision Tree

Every tree has the same 4 components — learn these first



Root Node	The starting point. First and most important question. One per tree.
Internal Node	A decision point (not the start). Splits data further.
Leaf Node	End point. No more splitting. This is the PREDICTION.
Branch	A path from parent to child based on YES or NO answer.

Gini Impurity — How the Tree Decides to Split

The mathematical tool CART uses to find the best question at each node

$$Gini(node) = 1 - \Sigma(p_i)^2$$

where p_i = proportion of class i in the node

Pure Node — 0.0

50 survived
0 not survived

$p_1=1.0,\ p_2=0.0$
$Gini = 1-(1^2+0^2) = 0.0$

✓ Perfect!
No impurity.

Mixed Node — 0.5

25 survived
25 not survived

$p_1=0.5,\ p_2=0.5$
$Gini = 1-(0.5^2+0.5^2) = 0.5$

✗ Worst case!
Maximum impurity.

Partial — 0.32

40 survived
10 not survived

$p_1=0.8,\ p_2=0.2$
$Gini = 1-(0.8^2+0.2^2) = 0.32$

◆ Acceptable.
Mostly one class.

CART Algorithm: For EVERY possible split → compute Gini for left & right child → pick the split with LOWEST weighted average Gini

Manual Tree Construction — The Bank Loan Example

CART algorithm step-by-step with 7 clients

ID Client	Debt	Job	Salary (k€)	Decision
1	Low	Stable	85	✅ 1 (Accept)
2	High	Stable	90	❌ 0 (Refuse)
3	Low	Unstable	45	❌ 0 (Refuse)
4	Low	Stable	120	✅ 1 (Accept)
5	High	Unstable	60	❌ 0 (Refuse)
6	Low	Stable	110	✅ 1 (Accept)
7	Low	Stable	50	❌ 0 (Refuse)

Now we test each feature to find the split that reduces Gini the most →

Step 1: Root Node Gini

```
7 clients total:
  • 3 Accepted (1): clients 1, 4, 6
  • 4 Refused  (0): clients 2, 3, 5, 7

Gini_root = 1 – [(3/7)² + (4/7)²]
          = 1 – [0.184 + 0.327]
          = 1 – 0.510 ≈ 0.49

Interpretation: Gini ≈ 0.5 means
the group is very mixed.
We MUST split to reduce impurity!
```

Testing Every Split — Which Feature Wins?

CART tries every possible split. The one with the LOWEST weighted Gini wins.

Debt

High vs Low

```
High branch: clients 2,5 → 0
Accepted, 2 Refused
  Gini_high = 1−(0²+1²) = 0.0  ✓
Pure!

Low branch: clients 1,3,4,6,7 → 3
Accept, 2 Refuse
  Gini_low = 1−[(3/5)²+(2/5)²] =
0.48

Weighted Cost = (2/7)×0 +
(5/7)×0.48 = 0.34
```

Cost = 0.34

Job

Stable vs Unstable

```
Unstable branch: clients 3,5 → 0
Accepted, 2 Refused
  Gini = 0.0  ✓ Pure!

Stable branch: clients 1,2,4,6,7 →
3 Accept, 2 Refuse
  Gini = 0.48

Weighted Cost = (2/7)×0 +
(5/7)×0.48 = 0.34
```

Cost = 0.34

Salary

< 70k vs ≥ 70k

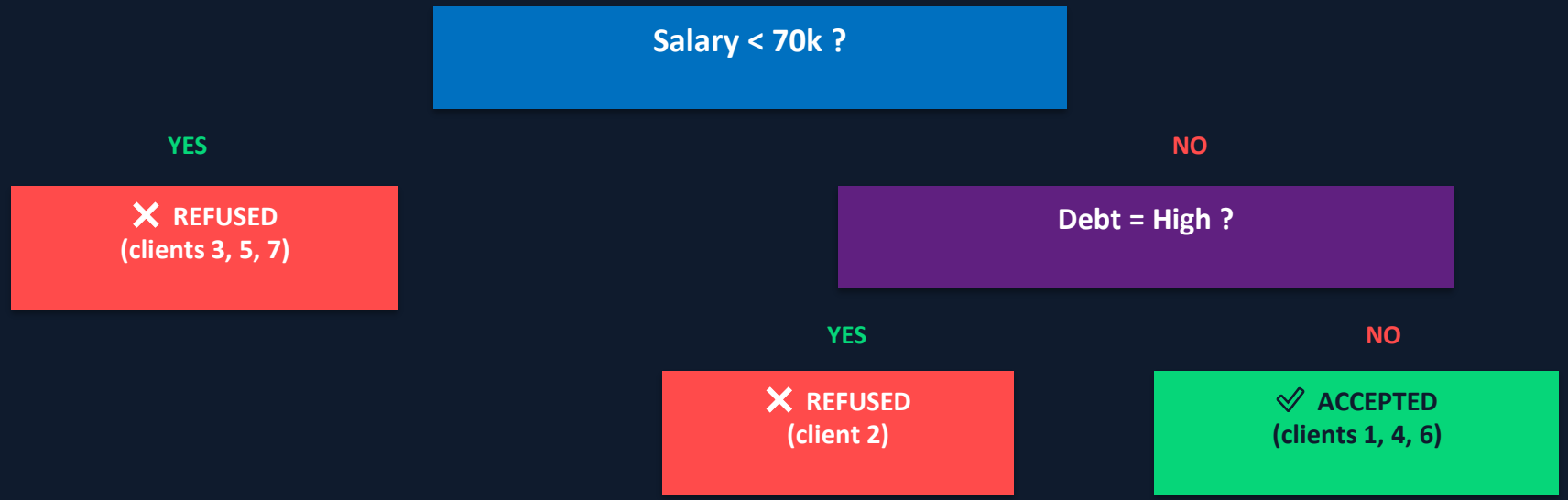
```
< 70k branch: clients 3,5,7 → 0
Accepted, 3 Refused
  Gini = 1−(0²+1²) = 0.0  ✓ Pure!

≥ 70k branch: clients 1,2,4,6 → 3
Accept, 1 Refuse
  Gini = 1−[(3/4)²+(1/4)²] = 0.375

Weighted Cost = (3/7)×0 +
(4/7)×0.375 = 0.21
```

Cost = 0.21 🏆 WINNER!

Salary < 70k? wins Round 1 with lowest cost (0.21) → becomes the ROOT node

The Final Decision Tree — Bank Loan

Three simple rules replace a complex dataset



The 3 Rules the Model Learned:

```
Rule 1: IF Salary < 70k THEN → REFUSED  (clients 3, 5, 7)
Rule 2: IF Salary ≥ 70k AND Debt = High THEN → REFUSED  (client 2)
Rule 3: IF Salary ≥ 70k AND Debt = Low  THEN → ACCEPTED  (clients 1, 4, 6)
```

Entropy & Information Gain — The ID3 Approach

An alternative impurity measure used by the ID3 algorithm

Entropy Formula

$$E(S) = -\Sigma\, p_i \times \log_2(p_i)$$

- p_i = proportion of class i
- High entropy = high disorder
- Entropy = 0 → perfectly pure
- Entropy = 1 → 50/50 split

Information Gain Formula

$$Gain(T,X) = E(T) - E(T,X)$$

- T = dataset, X = feature being tested
- Higher gain = feature reduces uncertainty more
- PICK the feature with HIGHEST info gain
- ID3 algorithm uses this (instead of Gini)

Play Golf Example: 9 Yes, 5 No → $E(PlayGolf) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$

Feature	Entropy After Split	Information Gain	Rank
Outlook	0.693	0.940 − 0.693 = 0.247	⍰ #1 Best
Humidity	0.788	0.940 − 0.788 = 0.152	⍰ #2
Windy	0.892	0.940 − 0.892 = 0.048	⍰ #3
Temperature	0.911	0.940 − 0.911 = 0.029	#4 Worst

Gini vs Entropy — Which Should You Use?

Both measure impurity — but they have different characteristics

Property	Gini Impurity	Entropy
Algorithm	CART (sklearn default)	ID3, C4.5
Range	0 to 0.5	0 to 1.0
Computation speed	Faster (no log math)	Slower (uses logarithm)
Result quality	Very similar outcomes	Very similar outcomes
Preferred for	Large datasets, speed	Multi-class problems
sklearn parameter	criterion='gini'	criterion='entropy'

Practical Advice: Use Gini (sklearn default) for speed. Both give very similar results. Don't overthink this choice!

Decision Tree Hyperparameters

These are the 'knobs' you control before training — they shape how the tree grows

`max_depth`

How many levels deep the tree can grow

Default: None (unlimited)

Recommended: 4–6 for most problems

Too deep = memorizes training data (overfitting). Too shallow = misses patterns (underfitting).

`min_samples_leaf`

Minimum passengers allowed in each leaf node

Default: 1 (any size allowed)

Recommended: 10–20 for Titanic

Small leaves memorize outliers. Larger leaves force broader, more generalizable rules.

`criterion`

Which formula to measure impurity at each split

Default: 'gini'

Recommended: 'gini' or 'entropy'

Gini is faster. Entropy sometimes gives slightly different trees. Both produce similar accuracy.

`random_state`

Controls randomness in the algorithm for reproducibility

Default: None (random)

Recommended: 42 (or any int)

Setting this ensures you get the SAME tree every time you run the code. Essential for teaching.

03

Overfitting & Why Trees Fail Alone

Overfitting — The #1 Enemy of Machine Learning

When a model memorizes instead of learning

Overfitting = the model performs VERY well on training data but POORLY on new test data

Underfitting (High Bias)

Model is too simple.
Doesn't learn enough.

```
Train accuracy: 60%
Test accuracy:  58%
→ Both low
```

Common Causes:

- max_depth too small
- K too large (KNN)

Just Right (Good Fit)

Model learned real patterns.
Generalizes to new data.

```
Train accuracy: 88%
Test accuracy:  85%
→ Both high, close together
```

Common Causes:

- Tune hyperparameters
- Cross-validate

Overfitting (High Variance)

Model memorized training.
Fails on test data.

```
Train accuracy: 99%
Test accuracy:  72%
→ Large gap!
```

Common Causes:

- max_depth unlimited
- min_samples_leaf=1

← Simpler model Complexity → Complexity goes up → risk of overfitting goes up

Why a Single Decision Tree is Not Enough

High variance means the tree changes drastically with small data changes

High Variance

A single tree is VERY sensitive to the training data. Add or remove just one passenger, and the tree structure can change completely. This makes predictions unstable.

Greedy Algorithm

The tree makes the BEST LOCAL decision at each split, but doesn't plan ahead. It might pick a good split now that leads to poor splits later — missing the globally optimal tree.

Memorization

Without depth limits, a tree memorizes every training example (Gini=0 at every leaf). It has zero ability to generalize to new passengers — 100% train, 70% test accuracy.

Sensitivity to Outliers

One unusual passenger can create a branch just for them. This branch is useless for future predictions — it's just noise learned as signal.

Solution: Use MANY diverse trees instead of one → Ensemble Methods (Random Forest, XGBoost)

04

Ensemble Learning

Ensemble Learning — Wisdom of the Crowd

"No one model is perfect — but many imperfect models together can be excellent"

Ensemble = Combine multiple models → reduce errors, improve accuracy, increase stability

🧤 Bagging (Bootstrap Aggregating)

Build many models IN PARALLEL on random subsets of data. Average/vote their predictions.

→ Reduces VARIANCE (overfitting)
→ Example: Random Forest

🚀 Boosting (Sequential Learning)

Build models SEQUENTIALLY. Each new model focuses on fixing the errors of the previous one.

→ Reduces BIAS (underfitting)
→ Examples: XGBoost, AdaBoost, GBM

📚 Stacking (Meta-Learning)

Train diverse base models, then train a meta-model to combine their predictions intelligently.

→ Combines strengths of different algorithms
→ Used in Kaggle competitions

Why does combining models work? → Error Diversification: models trained differently make DIFFERENT mistakes.
If Tree A is wrong about passenger X, Tree B might be right. Averaging cancels out individual errors.

Bagging — Bootstrap Aggregating Explained

How do you create many DIFFERENT trees from the SAME dataset?

Step 1 **Bootstrap Sampling**

Take the original dataset (N rows). Sample N rows WITH REPLACEMENT. Some rows appear 2–3 times; others are left out (~37% of data not selected = 'out-of-bag' samples).

Step 2 **Train Independent Trees**

Build a full Decision Tree on each bootstrapped sample. Each tree is trained in isolation — they don't know about each other. Can be parallelized on multiple CPU cores.

Step 3 **Aggregate Predictions**

For classification: take the MAJORITY VOTE across all trees. For regression: take the AVERAGE across all trees. Result is much more stable than any single tree.

Out-of-Bag (OOB) Validation: The ~37% of data NOT selected for each tree can be used as an automatic validation set — no need for a separate test split!

05

Random Forest

Random Forest — Bagging + Feature Randomness

Random Forest = Bagging + one extra trick: random feature selection at each split

Bootstrap Sample 1	Bootstrap Sample 2	Bootstrap Sample N
↓ Random features	↓ Random features	↓ Random features
Tree 1 → Survived	**Tree 2** → Not Survived	**Tree 3** → Survived

↓ Majority Vote: 2 Survived vs 1 Not Survived

✓ **FINAL: Survived**

The Secret Ingredient — Feature Randomness:

At each node split, only a RANDOM SUBSET of features is considered (√n features by default). This forces trees to be different from each other, which is what makes the ensemble powerful. Without this, all trees would be similar and the ensemble wouldn't help much.

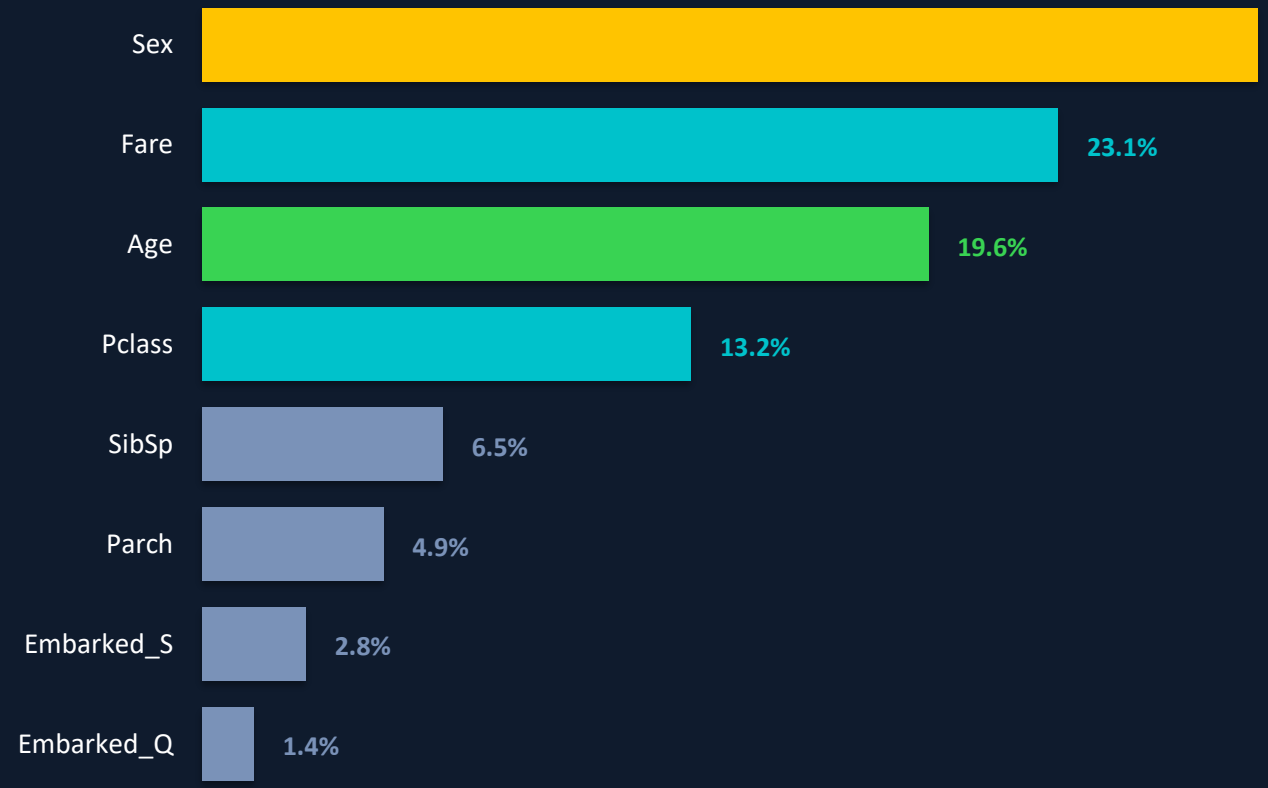
Random Forest — Key Hyperparameters

What you tune to control how the forest grows

Parameter	Default	Recommended	Why It Matters
`n_estimators`	100	100–500	More trees = more stable predictions. Diminishing returns after ~200. Increases training time.
`max_depth`	None	5–10	Shallow trees reduce overfitting per tree. Forest averaging compensates for simplicity.
`min_samples_leaf`	1	5–20	Larger values prevent noisy, tiny leaf nodes. Helps with overfitting on small datasets.
`max_features`	'sqrt'	'sqrt' or 'log2'	sqrt(n_features) is the proven default. Reduces feature correlation between trees.
`random_state`	None	42	Set this so your results are consistent every run. Essential for teaching and debugging.

Feature Importance — What Did the Forest Learn?

Random Forest tells you which features matter most for predictions



What This Tells Us:

Sex (28.5%):
Most important — gender was the #1 survival factor

Fare (23.1%):
Price = wealth = access to lifeboats

Age (19.6%):
Children first policy shows clearly in data

Embarked (4.2%):
Very low importance → Could be removed

Job was IGNORED in our bank tree — same idea!

`rf.feature_importances_ → returns an array of importance scores (sum = 1.0)`

Random Forest — Strengths & Weaknesses

Know these before you choose your model

✓ Strengths

- **Resistant to overfitting**
  Averaging many trees cancels out individual errors
- **No feature scaling needed**
  Tree splits use thresholds, not distances
- **Handles missing data**
  OOB samples provide automatic cross-validation
- **Feature importance**
  Built-in ranking of which features matter
- **Works out of the box**
  Good default settings, minimal tuning needed
- **Handles both types**
  Classification AND regression with same code
- **Parallel training**
  Each tree is independent — use all CPU cores

✗ Weaknesses

- **Not interpretable**
  You CAN'T read 100 trees like you can read 1 tree
- **Slow for large data**
  100+ trees × millions of rows = slow prediction
- **Memory intensive**
  Stores all N trees in memory simultaneously
- **Can still overfit**
  With very noisy features, the forest can still struggle
- **Extrapolation fails**
  Trees can't predict values outside training range

Rule of thumb: Start with Random Forest. If you need better accuracy, switch to XGBoost.

06

XGBoost — Extreme Gradient Boosting

XGBoost — Extreme Gradient Boosting

The algorithm that dominates data science competitions worldwide

Key Idea: Build trees SEQUENTIALLY — each new tree focuses on correcting the errors of all previous trees

Tree 1 (shallow)
Predicts using all features
89 errors remaining

→

Tree 2 (shallow)
Corrects Tree 1's worst mistakes
52 errors remaining

→

Tree 3 (shallow)
Corrects Trees 1+2 mistakes
34 errors remaining

→

Tree N (N=100)
Keeps correcting residual errors
~18 errors remaining

Final Prediction = Weighted Sum of ALL 100 Trees → Much more accurate than any single tree!

Why is this called GRADIENT Boosting? Each new tree is trained on the RESIDUALS (errors) of the previous predictions.
This is equivalent to performing gradient descent in function space — the model 'descends' toward lower error.
XGBoost adds: regularization (L1/L2), second-order optimization, parallel computation → faster and more accurate than basic GBM.

XGBoost — Hyperparameters Deep Dive

Understanding what each parameter controls and why it matters

Parameter	Default	Recommended	Why It Matters
n_estimators	100	100–300	More trees = better, but slower. Watch for overfitting past ~500.
max_depth	6	3–5	Keep trees SHALLOW (3–4 levels). Boosting works best with simple 'weak learners'.
learning_rate	0.3	0.05–0.1	Lower = more careful correction. Lower rate + more trees → usually better accuracy.
subsample	1.0	0.7–0.9	Using less than 100% adds randomness → prevents overfitting. Similar to bagging.
colsample_bytree	1.0	0.7–0.9	Similar to Random Forest's feature randomness. Reduces correlation between trees.
reg_alpha/lambda	0/1	Tune via CV	Penalizes complex models → reduces overfitting. Lambda is usually more useful.

Bagging vs Boosting — Side by Side

Two fundamentally different approaches to combining trees

Aspect	Bagging (Random Forest)	Boosting (XGBoost)
How trees are built	In PARALLEL (independently)	SEQUENTIALLY (each builds on last)
What each tree sees	Random bootstrap sample	Residuals / errors of previous trees
What it fixes	High Variance (overfitting)	High Bias (underfitting)
Speed (training)	Fast — parallel processing	Slower — must be sequential
Overfitting risk	Low (averaging helps)	Higher — must tune carefully
Accuracy (typical)	Very good	Often slightly better than RF
Interpretability	Low (forest)	Low (gradient path)
When to use	First model, noisy data	Maximum accuracy needed

💡 Practical Strategy: Start with Random Forest (quick, reliable). Squeeze extra accuracy with XGBoost. Use CV to compare.

Learning Rate — The Most Important XGBoost Parameter

Controls how aggressively each new tree corrects the residual errors

`learning_rate = 0.3` (Too High)

Aggressive corrections.
Big steps toward lower error.

+ Trains fast (few trees)

– Can overshoot the minimum

– Model is unstable

– Often overfits

`learning_rate = 0.1` (Standard)

Balanced correction.
Medium steps.

+ Good balance speed/accuracy

+ sklearn/XGBoost default

+ Works for most problems

Typical: 100 trees needed

`learning_rate = 0.01` (Too Low)

Very careful corrections.
Tiny steps.

+ Very accurate (if patient)

– Needs MANY more trees (1000+)

– Very slow to train

Use with large n_estimators

Golden Rule: Lower learning_rate + More n_estimators = Better accuracy (but slower). lr=0.05 to 0.1 with 200-500 trees is a common sweet spot.

07

Model Comparison & Selection

All 4 Models — Side by Side Comparison

Based on the Titanic dataset (25% test set, 223 passengers)

Model	Accuracy	Precision	Recall	F1-Score	Needs Scaling?
KNN (K=5)	78.9%	74.1%	73.0%	73.6%	✓ Yes
Decision Tree	80.7%	76.2%	74.2%	75.2%	✗ No
Random Forest	81.6%	80.8%	70.8%	75.4%	✗ No
XGBoost	82.5%	83.8%	69.7%	76.1%	✗ No

Winner: XGBoost (F1=76.1%) | Best Recall: Decision Tree | Best Precision: XGBoost

How to Choose the Right Model

There is no 'best' algorithm — it depends entirely on your situation

Data Size	If: Small (<1000 rows) → **KNN or Decision Tree**	If: Large (>10k rows) → **Random Forest or XGBoost**
Interpretability	If: Need to explain model → **Decision Tree (visual)**	If: Only accuracy matters → **XGBoost or RF**
Speed	If: Need fast training → **Random Forest (parallel)**	If: Can wait for results → **XGBoost**
Feature Scaling	If: No time to scale → **Any tree-based model**	If: Willing to scale → **KNN (requires scaling)**
Priority Metric	If: High Recall critical → **Tune threshold down**	If: High Precision needed → **Tune threshold up**
First attempt	If: Always start here → **Random Forest → reliable**	If: Need better accuracy → **XGBoost → powerful**

Reading the Classification Report — Titanic XGBoost

Let's read a real sklearn output and understand every number

```
                  precision    recall  f1-score   support

 Did Not Survive     0.8188    0.9104    0.8622       134
        Survived     0.8378    0.6966    0.7607        89

        accuracy                         0.8251       223
       macro avg     0.8283    0.8035    0.8115       223
    weighted avg     0.8264    0.8251    0.8217       223
```

precision = 0.8378 → Of 100 passengers we predicted 'Survived', 83.8 actually did

recall = 0.6966 → Of 89 actual survivors, we correctly identified 69.7% of them

f1-score = 0.7607 → Harmonic mean of 0.84 precision and 0.70 recall

support = 89 → There are 89 actual survivors in our test set of 223

accuracy = 0.8251 → 82.5% overall correct predictions (182/223)

macro avg → Simple average across both classes (treats both equally)

08

Summary & Cheat Sheet

All Formulas — Quick Reference Card

Everything you need to know on one slide

Gini Impurity

$$Gini(node) = 1 - \Sigma(p_i)^2$$

0 = pure, 0.5 = max impurity

Gini Cost Function

$$Cost = (N_left/N) \times Gini_L + (N_right/N) \times Gini_R$$

Pick lowest cost split

Entropy

$$E(S) = -\Sigma\ p_i \times \log_2(p_i)$$

0 = pure, 1 = max impurity

Information Gain

$$Gain(T,X) = E(T) - E(T,X)$$

Pick highest gain feature

Accuracy

$$Accuracy = (TP + TN) / Total$$

Overall correct predictions

Precision

$$Precision = TP / (TP + FP)$$

Quality of positive predictions

Recall

$$Recall = TP / (TP + FN)$$

Coverage of actual positives

F1-Score

$$F1 = 2 \times (P \times R) / (P + R)$$

Harmonic mean: balanced metric

Key Concepts — Final Summary

Overfitting

Model memorizes training data. Train accuracy >> Test accuracy. Fix: max_depth, min_samples_leaf, more data.

Underfitting

Model too simple to learn patterns. Both train and test accuracy are low. Fix: increase depth, reduce constraints.

Bagging

Build N trees in PARALLEL on random data subsets. Average/vote predictions. Reduces VARIANCE. → Random Forest.

Boosting

Build N trees SEQUENTIALLY. Each fixes previous errors. Reduces BIAS. → XGBoost, AdaBoost, GBM.

Feature Importance

Random Forest tracks how much each feature reduces Gini across all trees. High score = crucial feature.

Bootstrap Sampling

Sample N rows WITH replacement from dataset. ~63% unique rows, ~37% not selected (OOB = free validation set).

Data Leakage

Accidentally using test data during training (e.g., scaling on all data). Makes results unrealistically high.

Stratified Split

Ensures train and test have the SAME class ratios. Essential for imbalanced classification problems.

Code Cheat Sheet — sklearn & XGBoost

Copy-paste ready patterns for every model in this session

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=4,
min_samples_leaf=10, random_state=42)
dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)
```

XGBoost

```
from xgboost import XGBClassifier

xgb = XGBClassifier(n_estimators=100, max_depth=3,
learning_rate=0.1, random_state=42)
xgb.fit(X_train, y_train)

y_pred = xgb.predict(X_test)
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100,
max_depth=5, random_state=42)
rf.fit(X_train, y_train)

importances = pd.Series(rf.feature_importances_,
index=X.columns)
```

Evaluation

```
from sklearn.metrics import classification_report,
confusion_matrix
print(classification_report(y_test, y_pred, digits=4))

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

Real-World Applications of Ensemble Models

These algorithms power critical decisions in many industries

Banking & Finance

Credit Scoring (loan decisions), Fraud detection, Anti-money laundering, Stock price prediction

XGBoost is #1 algorithm used by banks for credit scoring. Handles thousands of features efficiently.

Transportation

Self-driving cars (sensor fusion), Traffic prediction, Route optimization, Insurance risk scoring

Decision Trees in self-driving systems: fast enough to run in real-time (milliseconds per prediction).

Healthcare

Disease diagnosis, Drug discovery, Patient readmission prediction, Medical image analysis

Random Forest identifies which genes are most predictive for cancer. F1-Score critical — missing a case is dangerous.

Technology

Spam detection, Ad click prediction, User behavior modeling, Content moderation

Gmail spam filter uses ensemble methods. Has to balance precision (real emails) vs recall (blocking spam).

E-Commerce

Product recommendations, Customer churn prediction, Price optimization, Inventory management

Amazon uses ensemble models to predict what you'll buy next. XGBoost won the famous Netflix prize competition.

Government

Tax fraud detection, Crime prediction, Social benefit eligibility, Environmental monitoring

IRS uses Random Forest to flag suspicious tax returns. XGBoost won Kaggle competitions on poverty prediction.

The Bias-Variance Tradeoff — Core Theory

Understanding this is the key to becoming a great ML practitioner

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Bias — Underfitting

The model's systematic ERROR — how far off its average prediction is from the truth.

High Bias means the model is too SIMPLE to capture the real patterns.

Causes:
- max_depth too small
- Algorithm too simple (e.g., KNN with large K)

Fix:
- Increase complexity
- Reduce regularization
- Use more features

Variance — Overfitting

How much the model's prediction CHANGES when trained on different data.

High Variance means the model is too SENSITIVE to training data — memorizes noise.

Causes:
- max_depth unlimited
- min_samples_leaf = 1
- Too many features

Fix:
- Add regularization (max_depth)
- More data
- Ensemble methods

Ensemble methods WIN: Bagging reduces VARIANCE. Boosting reduces BIAS. Together → state-of-the-art accuracy.

Preventing Overfitting — The Full Toolkit

Every technique below helps your model generalize to new data

Limit Tree Depth

`max_depth=4`

Most effective single fix. Prevents the tree from learning noise.

Cross-Validation

`cv=5 in GridSearchCV`

5-fold CV: train 5 times on different splits → more reliable accuracy.

Min Samples per Leaf

`min_samples_leaf=10`

Each leaf must represent at least N passengers, not just 1 outlier.

Regularization

`reg_alpha, reg_lambda`

XGBoost: penalizes complex models mathematically. L1=sparse, L2=smooth.

Use Ensemble Methods

`RandomForest, XGBoost`

Averaging 100 trees cancels out individual overfitting.

More Training Data

`Collect more data`

More examples → harder to memorize, forces learning of real patterns.

Train/Test Split

`test_size=0.25`

Always evaluate on data the model has NEVER seen. Non-negotiable.

Feature Selection

`Drop low importance`

Remove irrelevant features that add noise but no signal.

Practical ML Workflow — Step by Step

The professional workflow every data scientist follows

1 Define the Problem

What are you predicting? What metric matters? (Precision? Recall? F1?)
→ In Titanic: Binary classification. F1-Score because slightly imbalanced.

2 Prepare the Data

Clean, encode, scale, split train/test with stratify=y.
→ NEVER fit scaler on test data. No data leakage. stratify ensures balanced splits.

3 Build a Baseline

Train the simplest possible model first (Decision Tree, KNN).
→ Gives you a performance floor. Everything else should beat this.

4 Try Ensemble Methods

Random Forest → Quick, reliable, interpretable feature importance.
→ Usually 3-5% accuracy improvement over single tree with minimal tuning.

5 Optimize with XGBoost

Tune learning_rate, n_estimators, max_depth. Use cross-validation.
→ GridSearchCV automates hyperparameter search across all combinations.

6 Evaluate & Compare

Use classification_report + confusion matrix for ALL models.
→ Compare F1-Scores. Pick winner. Analyze where it fails (FN vs FP).

Common Mistakes — Avoid These!

Every beginner makes these — now you know better

✕ Fitting scaler on all data

scaler.fit_transform(X) THEN split → test data leaks into training!
Fix: Split FIRST, then fit scaler on X_train only → scaler.fit_transform(X_train) + .transform(X_test)

✕ Not setting random_state

Results change every run → impossible to debug or compare models reliably.
Fix: Always set random_state=42 on every model, split, and sampling operation.

✕ No stratify in train_test_split

Without stratify=y, your test set might have a very different class ratio than training.
Fix: Always use train_test_split(X, y, stratify=y) for classification problems.

✕ Unlimited tree depth

DecisionTreeClassifier() with no max_depth = 100% train accuracy, poor test accuracy.
Fix: Always set max_depth=4-6 and min_samples_leaf=10 for Titanic-sized datasets.

✕ Using accuracy on imbalanced data

If 95% of passengers didn't survive, accuracy of 95% is trivial (predict all '0').
Fix: Use F1-Score, Precision, Recall. Check confusion matrix for full picture.

✕ Comparing models on training data

A model with 99% TRAINING accuracy might have 70% TEST accuracy.
Fix: ALWAYS compare models using their performance on the TEST set (unseen data).

THANK YOU

FOR YOUR ATTENTION